



이웃 - 풀이

작성자: 나정휘

부분문제 1

$K_1, K_2 \geq 1$ 이므로 1번 학생과 2번 학생은 항상 이웃이다. 따라서 답은 항상 1 1 이다.

부분문제 2

$K_1 = K_2 = 1$ 이므로 1번 학생과 N 번 학생의 이웃은 1명, 다른 학생들의 이웃은 2명이다. 따라서 답은 항상 1 2 2 ... 2 1 형태이다.

부분문제 3

$1 \leq i, j \leq N$ 이고 $i \neq j$ 인 모든 정수 순서쌍 (i, j) 에 대해, $|i - j|$ 가 K_1 이하라면, i 번째 학생의 이웃 수를 1 증가시키는 것을 통해 모든 학생의 이웃 수를 계산할 수 있다.

부분문제 4

부분문제 3과 같이 모든 학생들의 쌍을 순회하면서, $S_i = S_j$ 일 때는 $|i - j| \leq K_1$ 인지, $S_i \neq S_j$ 일 때는 $|i - j| \leq K_2$ 인지 확인하고 이웃의 수를 증가시키면 된다.



가위바위보 - 풀이

작성자: 이유찬

부분문제 1

$N \leq 3$ 이므로 가능한 모든 대결 순서를 직접 시뮬레이션해도 충분하다.

부분문제 2

모든 사람이 같은 카드를 가지고 있는 경우, 임의의 순서로 대결을 진행하면서 원하는 사람을 승자로 만들 수 있다. 따라서 모든 사람이 우승 후보이다.

카드의 종류가 두 가지인 경우, 두 카드 중 한쪽이 다른 쪽을 이긴다. 이기는 카드를 W , 지는 카드를 L 이라 하면, W 를 가진 사람은 모든 L 을 이길 수 있고 같은 W 끼리는 임의로 승자를 결정할 수 있으므로 우승 후보이다. 반대로 L 을 가진 사람은 W 를 이길 수 없으므로 우승할 수 없다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 3

부분문제 3의 풀이는 만점 풀이로 이어지지 않는다. 각 사람 i 에 대해 우승 가능 여부를 다음과 같은 구간 DP로 판정할 수 있다.

$\text{cards}[l][r]$ 을 구간 $[l, r]$ 의 사람들끼리만 대결을 진행했을 때, 마지막에 남을 수 있는 사람의 카드 종류의 집합으로 정의한다. DP의 전이는 분할점 k 에 대해 $\text{cards}[l][k]$ 의 어떤 카드와 $\text{cards}[k+1][r]$ 의 어떤 카드가 만나 살아남을 수 있는 카드를 모두 합치면 된다.

i 의 우승 가능 여부 $\text{win}[l][r][i]$ 도 유사하게 정의할 수 있다. 전이에서는 분할점 k 에 대해 i 가 한쪽 구간에서 살아남고, 반대쪽 구간의 어떤 카드와 만나 A_i 가 이기거나 같은 경우가 존재하는지 확인한다.

이를 그대로 구현하면 $\mathcal{O}(N^4)$ 정도가 되고, $N \leq 100$ 에서는 충분히 동작한다.

부분문제 4

A_i 를 이기는 카드를 Y , Y 를 이기는 카드를 Z 라 하자 (A_i 가 R 이면 $Y = P$, $Z = S$ 이고, 나머지도 대칭적으로 정의한다).

핵심 관찰: i 가 우승 가능할 필요충분조건은, i 의 왼쪽과 오른쪽 각각에 대해 Y 가 등장하지 않거나, Y 와 Z 가 모두 등장하는 것이다.

증명: 일반성을 잃지 않고 $A_i = R$ 이라 하자. i 가 우승하려면 결국 자기 자신을 제외한 모든 사람을 제거해야 하므로, i 의 왼쪽과 오른쪽 양쪽에 있는 사람들을 모두 처리해야 한다. 양쪽은 i 를 거치지 않고는 서로 상호작용할 수 없으므로 독립적으로 분석할 수 있다.

i 의 한쪽(편의상 왼쪽)에 대해 살펴보자.

- 왼쪽에 P가 없는 경우, 왼쪽 사람들의 카드는 R과 S로만 구성된다. i 자신이 R이므로 S를 모두 이길 수 있고, R끼리는 임의로 승자를 정할 수 있어 왼쪽을 모두 정리할 수 있다.
- 왼쪽에 P가 있지만 S가 없는 경우, 왼쪽 사람들의 카드는 R과 P로만 구성된다. S가 없으므로 어떤 P도 제거할 수 없다. 따라서 i 는 우승할 수 없다.
- 왼쪽에 P와 S가 모두 있는 경우, S를 이용해 모든 P를 제거할 수 있다. S에 인접한 P는 S로 직접 이겨 제거하고, S와 P 사이에 R이 끼어 있으면 P로 R을 먼저 제거한 뒤 다시 S로 P를 제거하면 된다. 이 과정을 반복하면 모든 P를 없앨 수 있고, 남은 R과 S는 i 가 직접 모두 이길 수 있다.

오른쪽도 대칭적으로 분석할 수 있다.

이 관찰을 각 i 마다 직접 왼쪽과 오른쪽을 확인하면, 각 i 마다 $O(N)$ 이 들어 전체 시간 복잡도는 $O(N^2)$ 이다.

부분문제 5

부분문제 4의 판정을 각 i 에 대해 $O(1)$ 에 수행하기 위해, 각 카드 종류별로 정방향/역방향 누적 등장 횟수를 미리 계산해 둔다. 그러면 i 의 왼쪽/오른쪽 각각에서 특정 카드가 등장하는지를 $O(1)$ 에 알 수 있다.

전체 시간 복잡도는 $O(N)$ 이다.

수열 정렬하기 - 풀이

작성자: 장근영

기본 관찰

한 번의 시행에서 기준값 x 를 고르면, 수열은 $[A_i \leq x]$ 와 $[A_i > x]$ 를 이어 붙인 형태가 된다. 여기서 각각의 부분수열 안에서는 원래 수열에서의 상대적인 순서가 유지된다.

여러 번의 시행에서 선택한 기준값들을 생각하자.

먼저 같은 기준값을 여러 번 선택하는 것은 의미가 없다. 한 번 x 를 기준으로 나누고 나면, 이미 모든 x 이하의 원소가 모든 x 초과 원소보다 앞에 있으므로, 다시 같은 x 를 선택해도 수열은 변하지 않는다.

또한 서로 다른 두 기준값 x, y 에 대해, 이 둘을 적용하는 순서도 중요하지 않다. $x < y$ 라고 하자.

먼저 x 를 적용한 뒤 y 를 적용하면 다음과 같다. x 를 적용한 뒤 수열은 $[A_i \leq x] + [A_i > x]$ 가 된다. 이제 y 를 적용하면, 앞쪽의 $[A_i \leq x]$ 에 속한 원소들은 모두 y 이하이므로 그대로 앞에 남는다. 뒤쪽의 $[A_i > x]$ 중에서는 $x < A_i \leq y$ 인 원소들이 그다음으로 오고, $y < A_i$ 인 원소들이 마지막에 온다. 따라서 최종 수열은 $[A_i \leq x] + [x < A_i \leq y] + [y < A_i]$ 가 된다.

반대로 먼저 y 를 적용한 뒤 x 를 적용해도 같은 결과가 나온다. y 를 적용한 뒤 수열은 $[A_i \leq y] + [A_i > y]$ 가 된다. 이제 x 를 적용하면, 앞쪽의 $[A_i \leq y]$ 안에서 $A_i \leq x$ 인 원소들이 먼저 오고, 그다음에 $x < A_i \leq y$ 인 원소들이 온다. 뒤쪽의 $[A_i > y]$ 에 속한 원소들은 모두 x 초과이므로 마지막에 남는다. 따라서 이 경우에도 최종 수열은 $[A_i \leq x] + [x < A_i \leq y] + [y < A_i]$ 가 된다.

각 단계에서 같은 범위에 속한 원소들끼리는 원래 수열에서의 상대적인 순서가 계속 유지된다. 따라서 두 기준값 x, y 를 적용한 결과는 적용 순서와 무관하다.

결국 여러 번의 시행에서 선택한 기준값들은 순서를 바꾸어도 최종 결과가 변하지 않는다. 따라서 선택한 기준값들을 중복 없이 정렬해서 $x_1 < x_2 < \dots < x_k$ 라고 두어도 된다.

이제 이 기준값들을 오름차순으로 적용한다고 생각하자. 처음 x_1 을 적용하면 수열은 $[A_i \leq x_1] + [x_1 < A_i]$ 가 된다. 그다음 x_2 를 적용하면 첫 번째 부분 $[A_i \leq x_1]$ 은 모두 x_2 이하이므로 그대로 앞에 남고, 뒤쪽 부분 $[x_1 < A_i]$ 이 $[x_1 < A_i \leq x_2]$ 와 $[x_2 < A_i]$ 로 나뉜다.

이 과정을 반복하면, 최종 수열은 $[A_i \leq x_1], [x_1 < A_i \leq x_2], \dots, [x_{k-1} < A_i \leq x_k], [x_k < A_i]$ 를 차례대로 이어 붙인 수열이 된다.

즉, 시행들의 순서는 중요하지 않고, 선택한 기준값들의 집합만 중요하다. 따라서 문제는 값의 범위 $[1, N]$ 을 몇 개의 연속한 구간으로 나누어서, 각 구간에 속한 값들만 원래 순서대로 뽑은 부분수열이 비내림차순이 되게 만드는 문제로 볼 수 있다.

값의 구간 수가 c 개라면, 그 구간들을 만들기 위해 필요한 시행 횟수는 $c - 1$ 번이다.

부분문제 1

모든 원소가 1 또는 2이다.

이미 모든 1이 모든 2보다 앞에 있다면 수열은 비내림차순이므로 답은 0이다.

그렇지 않다면 어떤 2가 어떤 1보다 앞에 있다. 이 경우 $x = 1$ 로 한 번 시행하면 모든 1이 앞으로 가고, 모든 2가 뒤로 가므로 수열이 정렬된다.

따라서 수열에 2가 1보다 앞에 등장하는 경우가 있으면 답은 1, 그렇지 않으면 답은 0이다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 2

$N \leq 15$ 이다.

의미 있는 기준값은 $1, 2, \dots, N - 1$ 뿐이다. 따라서 가능한 기준값의 부분집합을 모두 시도할 수 있다.

각 부분집합 S 에 대해, S 의 원소들을 오름차순으로 적용한 결과를 직접 만들어 본다. 그 결과가 비내림차순이면, $|S|$ 가 가능한 시행 횟수이다.

모든 부분집합을 확인하면서 가능한 $|S|$ 의 최솟값을 구하면 된다.

부분집합의 개수는 최대 2^{N-1} 개이고, 각 경우에 대해 시행을 직접 시뮬레이션하면 충분하다.

전체 시간 복잡도는 $\mathcal{O}(2^N N^2)$ 이다.

부분문제 3

$N \leq 100$ 이다.

값의 구간 $[l, r]$ 에 대해, 값이 l 이상 r 이하인 원소들만 원래 순서대로 뽑았을 때 그 부분수열이 비내림차순인지 확인하자. 이를 $\text{ok}(l, r)$ 라고 하자.

$\text{ok}(l, r)$ 를 가장 단순하게 계산하려면, 모든 위치 쌍 $p < q$ 를 확인하면 된다.

만약 $l \leq A_p \leq r, l \leq A_q \leq r$ 이면서 $A_p > A_q$ 인 쌍이 존재한다면, 구간 $[l, r]$ 만으로 이루어진 부분수열은 비내림차순이 될 수 없다. 그런 쌍이 없다면 가능하다.

하나의 (l, r) 에 대해 모든 위치 쌍을 확인하는 데 $\mathcal{O}(N^2)$ 시간이 걸린다. 가능한 (l, r) 쌍은 $\mathcal{O}(N^2)$ 개이므로, 모든 $\text{ok}(l, r)$ 를 계산하는 데 $\mathcal{O}(N^4)$ 시간이 걸린다.

이제 DP를 하자. $\text{dp}[i]$ 를 값 $1, 2, \dots, i$ 를 몇 개의 연속한 값 구간으로 나누어야 하는지의 최솟값이라고 하자.

초기값은 $\text{dp}[0] = 0$ 이다.

마지막 구간이 $[j + 1, i]$ 라고 하면, 이 구간은 $\text{ok}(j + 1, i)$ 가 참이어야 한다. 따라서 $0 \leq j < i$ 인 모든 j 중에서 가능한 것을 골라 $\text{dp}[i] = \min(\text{dp}[j] + 1)$ 로 계산할 수 있다.

최소 시행 횟수는 구간 수보다 1 작으므로, 정답은 $\text{dp}[N] - 1$ 이다.

전체 시간 복잡도는 $\mathcal{O}(N^4)$ 이다.

부분문제 4

$N \leq 750$ 이다.

부분문제 3과 같은 DP를 사용하되, $ok(l, r)$ 를 더 빠르게 계산한다.

값의 구간 $[l, r]$ 에 대해, 수열 A 를 왼쪽에서 오른쪽으로 한 번 훑는다. 이때 $l \leq A_i \leq r$ 인 원소만 보면서, 직전에 선택된 값보다 현재 값이 작아지는 순간이 있는지 확인한다.

현재까지 선택한 마지막 값을 **last**라고 하자. $l \leq A_i \leq r$ 인 원소를 만났을 때 **last** $> A_i$ 라면, $ok(l, r)$ 는 거짓이다. 그렇지 않다면 **last**를 A_i 로 갱신한다.

이렇게 하면 하나의 (l, r) 에 대해 $ok(l, r)$ 를 $\mathcal{O}(N)$ 시간에 계산할 수 있다.

가능한 (l, r) 쌍은 $\mathcal{O}(N^2)$ 개이므로, 모든 $ok(l, r)$ 를 계산하는 데 $\mathcal{O}(N^3)$ 시간이 걸린다.

이후 DP는 부분문제 3과 동일하다. $dp[0] = 0$ 으로 두고, 마지막 구간 $[j + 1, i]$ 가 가능한 모든 j 에 대해 $dp[i] = \min(dp[j] + 1)$ 을 계산한다.

정답은 $dp[N] - 1$ 이다.

전체 시간 복잡도는 $\mathcal{O}(N^3)$ 이다.

부분문제 5

모든 원소가 서로 다르다.

수열의 길이가 N 이고, 모든 값이 1 이상 N 이하이므로, 수열 A 는 $1, 2, \dots, N$ 의 순열이다.

값 i 가 등장하는 위치를 p_i 라고 하자.

값 i 와 $i + 1$ 이 같은 값 구간에 들어간다고 하자. 같은 구간 안에서는 원래 수열에서의 상대적인 순서가 유지된다.

따라서 $p_i > p_{i+1}$ 이면, 원래 수열에서 $i + 1$ 이 i 보다 앞에 있으므로 i 와 $i + 1$ 을 같은 구간에 둘 수 없다. 즉, 반드시 i 와 $i + 1$ 사이를 나누는 기준값을 골라야 한다.

반대로 $p_i < p_{i+1}$ 이면, i 와 $i + 1$ 사이를 나누지 않아도 이 둘의 순서는 올바르다.

따라서 답은 $p_i > p_{i+1}$ 인 i 의 개수이다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이다.

부분문제 6

이제 추가 제약이 없다.

등장하는 서로 다른 값들을 오름차순으로 $v_1 < v_2 < \dots < v_M$ 이라고 하자.

각 값 v_i 에 대해, L_i 를 v_i 가 처음 등장하는 위치, R_i 를 v_i 가 마지막으로 등장하는 위치라고 하자.

인접한 두 등장값 v_i, v_{i+1} 를 생각하자.

만약 $R_i > L_{i+1}$ 이라면, 어떤 v_i 가 어떤 v_{i+1} 보다 뒤에 등장한다는 뜻이다. 즉, 원래 수열 안에 v_{i+1}, v_i 순서로 등장하는 두 원소가 존재한다.

이 둘이 같은 값 구간에 남아 있다면, 구간 내부의 상대적인 순서는 바뀌지 않으므로 비내림차순이 될 수 없다. 따라서 이 경우에는 반드시 v_i 와 v_{i+1} 사이를 나누어야 한다.

이때 기준값 x 는 $v_i \leq x < v_{i+1}$ 가 되도록 고르면 된다. 예를 들어 $x = v_i$ 를 고를 수 있다.

반대로 $R_i < L_{i+1}$ 이라면, 모든 v_i 가 모든 v_{i+1} 보다 앞에 등장한다. 따라서 v_i 와 v_{i+1} 를 같은 구간에 두어도 이 둘 사이의 순서는 문제가 없다.

그러므로 반드시 나누어야 하는 경계는 정확히 $R_i > L_{i+1}$ 인 경계들이다.

이 경계들을 모두 나누면, 각 값 구간 안에서는 인접한 등장값들 v_i, v_{i+1} 가 모두 $R_i < L_{i+1}$ 을 만족한다. 따라서 구간 안의 값들은 원래 수열에서도 작은 값부터 큰 값 순서로 등장하고, 그 구간의 부분수열은 비내림차순이다.

구간들 자체도 값의 범위가 증가하는 순서로 이어 붙여지므로, 전체 수열 역시 비내림차순이 된다.

따라서 정답은 $R_i > L_{i+1}$ 인 i 의 개수이다.

구현은 다음과 같이 하면 된다.

1. 각 값 x 에 대해 첫 등장 위치 $\text{first}[x]$, 마지막 등장 위치 $\text{last}[x]$ 를 계산한다.
2. 값 $1, 2, \dots, N$ 을 순서대로 보면서 실제로 등장한 값만 확인한다.
3. 직전에 등장한 값이 p , 현재 등장한 값이 x 일 때, $\text{last}[p] > \text{first}[x]$ 이면 답을 1 증가시킨다.

전체 시간 복잡도는 $\mathcal{O}(N)$ 이고, 메모리 복잡도는 $\mathcal{O}(N)$ 이다.